PRN: 260840125033 & 260840125041

## Assignment 2

**1. interface**
**For the hierarchy created in the previous example, implement the following requirements.**

**Create Interface InsurancePolicy with method calculatePremium()**

**- Each vehicle must be insured**

**- For cars if the car is luxury car then premium amount would be 15000 else 7000**
**- If bus capacity is > 12 then premium amount will be 18000 else 10000**

**The Policy app can be used to see InsuranceDue amount for any vehicle. Implement a method to display the policy.**

// Vehicle Class

```
package day5assignment2;

public abstract class Vehicle implements InsurancePolicy {
     protected int regno;
     protected static int count = 0;
     protected String brand;
     protected double rentalRate;

     public Vehicle(String brand, double rentalRate) {
     this.regno = ++count;
     this.brand = brand;
     this.rentalRate = rentalRate;
     }

     public abstract double calculateRent(int hours);
     public abstract double calculatePremium();
     @Override

     public String toString() {
     return "Vehicle [regno=" + regno + ", brand=" + brand +
     ", rentalRate=" + rentalRate + "]";
     }

     public int getRegno() {
     return regno;
     }

     public void setRegno(int regno) {
     this.regno = regno;
     }
```

```
     public String getBrand() {
     return brand;
     }

     public void setBrand(String brand) {
     this.brand = brand;
     }

     public double getRentalRate() {
     return rentalRate;
     }
     public void setRentalRate(double rentalRate) {
     this.rentalRate = rentalRate;
     }

}

// Bus Class
package day5assignment2;


public class Bus extends Vehicle {
     protected int seatingCapacity;

     public Bus(String brand, double rentalRate, int
seatingCapacity) {
          super(brand, rentalRate);
          this.seatingCapacity = seatingCapacity;
     }

     @Override
     public double calculateRent(int hours) {
          double rate;
          if (this.seatingCapacity < 12) {
               rate = hours * 50;
          } else {
               rate = hours * 70;
          }
          return rate;
     }

     @Override
     public double calculatePremium() {
          double amount;
          if(seatingCapacity > 12) {
               amount = 18000;
          }

          else {
               amount = 10000;
          }
          return amount;
```

```
	}


	public int getSeatingCapacity() {
		return seatingCapacity;
	}

	public void setSeatingCapacity(int seatingCapacity) {
		this.seatingCapacity = seatingCapacity;
	}

	@Override
	public String toString() {
		return "Bus [Brand: "+ this.brand + ", Rental Rate: "+
this.rentalRate + ", seatingCapacity " + seatingCapacity + "]";
	}
}
// Car Class
package day5assignment2;

public class Car extends Vehicle {
	protected boolean isLuxury;
	protected int hours;


	public Car(String brand, double rentalRate, boolean isLuxury)
	{
		super(brand,rentalRate);
		this.isLuxury=isLuxury;

	}

	@Override
	public double calculateRent(int hours) {
		double rate ;

		if(this.isLuxury ) {
			if(hours < 8) {
				rate = hours * 40;
			}
		else {
			rate = 40 * 8; // 8 hours
			hours -= 8;
			rate += hours * 50;
		}}

		else {
			if(hours < 8) {
				rate = hours * 25;
			}
			else {
				rate = 25 * 8; // 8 hours
```

```
                    hours -= 8;}
                    rate += hours * 35;
               }
               return rate;
          }

          @Override
          public double calculatePremium() {
               double amount;
               if(isLuxury == true) {
                    amount = 15000;
               }

               else {
                    amount = 7000;
               }
               return amount;

          }
          public boolean isLuxury() {
          return isLuxury;
          }

          public void setLuxury(boolean isLuxury) {
          this.isLuxury = isLuxury;
          }

          public int getHours() {
          return hours;
          }

          public void setHours(int hours) {
          this.hours = hours;
          }

          @Override
          public String toString() {
               return "Car [ Brand: "+ this.brand + ", Rental
Rate: "+ this.rentalRate + ", NameisLuxury= " + isLuxury + ",
hours=" + hours + "]";

          }
}// Insurance Policy
package day5assignment2;

public interface InsurancePolicy {
     double calculatePremium();


}
// Policy App
package day5assignment2;
```

```
public class PolicyApp {

	public static void main(String[] args) {
		Car c1 = new Car("Ferrari", 1000, false);
		Car c2 = new Car("Lamborghini", 800, true);
		Bus b1 = new Bus("Royalaman", 50, 30);
		Bus b2 = new Bus("CityBus", 20, 10);

		System.out.println(c1);
		System.out.println("Premium: " + c1.calculatePremium());
		System.out.println(c2);
		System.out.println("Premium: " + c2.calculatePremium());
		System.out.println(b1);
		System.out.println("Premium: " + b1.calculatePremium());
		System.out.println(b2);
		System.out.println("Preamanmium: " +
b2.calculatePremium());

	}

}
```

---

**2. Exception handling**
**For Account class hierarchy :**
**for the deposit and withdraw methods**
**Identify possible exceptions and create custom exception classes for the same**
**Display meaningful error messages**

```
// Account Class
package day2_30Sep;
import  class1.Date;



public class Account {
	private int accid;
	private String name;
	private double balance;
	private Date joiningdate;

//   static attributes are accessed directly using class name , we
don't need a object to access a static member
	private static float intrate;
	static {
		intrate=6.5f;
	}
	static {
```

```
            System.out.println("Block 2");
      }

      public Account(int accid,String name,double balance,Date
joiningdate) {
            this.accid=accid;
            this.name=name;
            this.balance=balance;
            this.joiningdate=joiningdate;
      }


      public void calculateInstrest() {
            double intrest= (balance*intrate*0.4);
            System.out.println("Interest: "+intrest);
      }

      public static void main(String[] args) {
            // TODO Auto-generated method stub

      }

      public String getName() {
            return name;
      }

      public void setName(String name) {
            this.name = name;
      }

      public int getAccid() {
            return accid;
      }

      public void setAccid(int accid) {
            this.accid = accid;
      }

      public double getBalance() {
            return balance;
      }

      public void setBalance(double balance) {
            this.balance = balance;
      }

}
```

// Invalid Amount Exception

```
package day5assignment2;

public class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
```

```
        super(message);
    }


// Insufficient Balance Exception

package day5assignment2;

public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}
```

---

## Self Learning Task

## Concepts of exceptions in Inheritance

### Checked Exceptions

1. If a parent method throws an exception, the child can override it . It means the child can throw same checked exception

Suppose:

```
class Parent {
    void show() throws Exception {
        System.out.println("Parent");
    }
}
```

Child overrides it:

```
class Child extends Parent {
    @Override
    void show() throws Exception {
        System.out.println("Child");
    }
}
```

2. Child can throw a MORE SPECIFIC exception such as IOException which is a subclass of Exception.

3. Child can throw an even narrower exception such as FileNotFoundException which is a subclass of IOException

4. Child can throw NO exception

```
class Parent {
    void show() throws Exception {
    }
}

class Child extends Parent {
    @Override
    void show() {
    }
}
```

5. But the Child cannot throw a broader checked exception.

6. Also If Parent doesn’t throw an exception then child can not throw checked exceptions.

7. Multiple exceptions

```
class Parent {
    void test() throws IOException, SQLException {
    }
}
```

Child can throw IOException or FileNotFoundException

## Unchecked Exceptions

Unchecked exceptions are subclasses of: RuntimeException

For Example:

```
RuntimeException
  ├── NullPointerException
  ├── ArithmeticException
  ├── ArrayIndexOutOfBoundsException
  └── IllegalArgumentException
```

Suppose:

```
class Parent {
    void show() throws IOException {
    }
}

class Child extends Parent {
```

```
    @Override
    void show() throws RuntimeException { // Also can throw
specific runtime exceptions.
    }
}
```

If Parent doesn’t throw an exception then child can throw unchecked exceptions.

**Overall Logic**

Overriding + checked exceptions = restrictions apply.
Overriding + unchecked exceptions = no such restriction.